

Python Programming with Artificial Intelligence

Machine Learning

Huangxun Chen @IoT&AI Thrust



Libraries for AI Workflows

- Numpy, SciPy, Pandas, Matplotlib & Seaborn

Machine Learning Basics

- Core Terminology, Main Categories, Core Workflow, Main Challenges



Libraries for AI Workflows

- Numpy
- SciPy
- Pandas
- Matplotlib & Seaborn

Connect Python to AI-related computation

- Numpy: fast N-D arrays, vectorization, linear algebra
- SciPy: algorithms on top of NumPy: optimize, stats, sparse, signal.
- Pandas: tabular data I/O, cleaning, transformation, grouping, joins.
- Matplotlib & Seaborn: publication-quality plots; communicate results.

Why they matters?

- **Performance** (vectorized C/Fortran kernels), **usability** (clear APIs), and **ecosystem** (works with scikit-learn / PyTorch).
- These are the foundations you will reuse in all AI/ML work.

Roles at a Glance

Library	Core Idea	Primary Data Type	Typical Tasks
NumPy	Vectorized numerical computing	ndarray	Element-wise ops, broadcasting, LA, random
SciPy	Algorithms on arrays	ndarray/sparse	Optimize, stats tests, sparse ops, FFT
pandas	Tabular data wrangling	DataFrame/Series	I/O, cleaning, merge/join, groupby, features
Matplotlib	Visualization backend	n/a	Line/scatter/bar, images, layout, export

Table: High-level comparison for when to reach for each library.



Numpy

Why Numpy?

Problem: Standard Python lists are flexible but slow for numerical operations.

- Loops in Python are inefficient for large-scale math due to interpreter overhead.
- Lists can store mixed data types, which prevents optimizations.

The Solution: NumPy (Numerical Python)

- Provides the powerful **N-dimensional array** (ndarray) object.
- Achieves its speed through a core principle called **Vectorization**.
- Operations are implemented in fast, compiled C/Fortran code.
- Foundation for libraries like Pandas, Scikit-learn, and TensorFlow.

```
1  # Let's sum the squares of 10 million numbers
2  # Python List
3  %timeit sum([x*x for x in range(10000000)]) # >> ~800 ms
4  # NumPy Array
5  import numpy as np
6  arr = np.arange(10000000)
7  %timeit np.sum(arr * arr) # >> ~15 ms (>50x faster!)
```

■ The Core Principle: Vectorization

Vectorization is the process of applying an operation to an entire array at once, rather than element-by-element in a Python loop.

Without Vectorization (Slow Python Loop)

```
1 c = []
2 for i in range(len(a)):
3     c.append(a[i] + b[i])
```

With Vectorization (Fast NumPy Operation)

```
1 c = a + b
```

Why is this faster?

- The loop is moved from Python (interpreted) to C (pre-compiled).
- It reduces overhead by performing a single operation on the whole data block.
- It can utilize modern CPU capabilities (like SIMD) to process multiple elements in parallel.

■ The Core Object: `numpy.ndarray`

The N-Dimensional Array is the heart of NumPy. It's a grid of values, all of the same type, which enables vectorization.

1D Array (Vector)

1	5	2	7
---	---	---	---

2D Array (Matrix)

1.0	2.0	3.0
4.0	5.0	6.0

Key Attributes:

`arr.ndim`: The number of dimensions (axes).

`arr.shape`: A tuple indicating the size of the array in each dimension.

`arr.size`: The total number of elements.

`arr.dtype`: The data type of the elements (e.g., `int64`, `float64`)

Summary of Common Operations

Category	Common Functions & Methods
Creation	<code>np.array</code> , <code>np.arange</code> , <code>np.zeros</code> , <code>np.ones</code> , <code>np.linspace</code> , <code>np.random.rand</code>
I/O	<code>np.save</code> , <code>np.load</code> , <code>np.savez_compressed</code>
Manipulation	<code>.reshape</code> , <code>.T</code> , <code>.flatten</code> , <code>np.concatenate</code> , <code>np.vstack</code> , <code>np.hstack</code> , <code>np.split</code>
Math (ufuncs)	<code>np.add</code> , <code>np.subtract</code> , <code>np.multiply</code> , <code>np.divide</code> , <code>np.sqrt</code> , <code>np.exp</code> , <code>np.sin</code>
Aggregation	<code>.sum</code> , <code>.mean</code> , <code>.std</code> , <code>.min</code> , <code>.max</code> , <code>.argmin</code> , <code>.argmax</code>
Linear Algebra	<code>np.dot</code> , <code>@</code> , <code>np.einsum</code> , <code>np.linalg.inv</code> , <code>np.linalg.eig</code>
Searching/Logic	<code>np.where</code> , <code>np.argsort</code> , <code>np.unique</code> , <code>np.in1d</code>
Indexing	Slicing <code>[:]</code> , boolean masking <code>[arr > 0]</code> , fancy indexing <code>[indices]</code>

Table: This is not exhaustive but covers the vast majority of common use cases.

1. From Python Lists

```
1 # Create a 1D array
2 a = np.array([1, 2, 3, 4])
3 # Create a 2D array and specify data type
4 m = np.array([[1, 2], [3, 4]], dtype=np.float64)
```

2. Using Built-in Functions (often more efficient)

```
1 # Create an array of zeros (shape 2x3)
2 zeros = np.zeros((2, 3))
3 # Create an array with a range of elements
4 seq = np.arange(0, 10, 2) # [0 2 4 6 8]
5 # Create an array of random values (uniform distribution)
6 rand_arr = np.random.rand(2, 2)
```

Other common creators: `np.full()`, `np.linspace()`, `np.eye()`

Saving & Loading Arrays

For any serious AI work, you need to save model weights, preprocessed data, or experiment results.

Binary Format (.npy)

- The most efficient way to save a single NumPy array

```
1 data = np.random.randn(100, 100)
2 np.save('my_data.npy', data)
3 loaded_data = np.load('my_data.npy')
```

Compressed Archive (.npz)

- Ideal for saving multiple arrays into a single, compressed file.

```
1 weights = np.array([0.5, 0.1, -0.2])
2 biases = np.array([1.2, 0.7])
3 np.savez_compressed('model.npz', w=weights, b=biases)
4 model = np.load('model.npz')
5 print(model['w']) # Access by keyword
```

Manipulating Array Shapes

Reshaping

- `arr.reshape(new_shape)` returns a view with a new shape.
- The total number of elements must remain the same

```
1 a = np.arange(12)          # Shape (12,)
2 b = a.reshape(3, 4)        # Shape (3, 4)
3 c = a.reshape(2, -1)       # Shape (2, 6), NumPy infers the
                             6
```

Adding a New Dimension

- Use `np.newaxis` to increase the dimension of an array.

```
1 v = np.array([1, 2, 3])    # Shape (3,)
2 col_v = v[:, np.newaxis]   # Shape (3, 1)
```

Transposing

- `arr.T` swaps the axes of an array (e.g., rows become columns).

Combining and Splitting Arrays

Concatenating and Stacking

- `np.concatenate([a, b], axis=...)` joins arrays along an axis.
- `np.vstack()` and `np.hstack()` are helpers for vertical and horizontal stacking.

```
1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6]])
3 # Concatenate along rows (axis 0)
4 print(np.concatenate([a, b], axis=0))
5 # [[1 2]
6 #   [3 4]
7 #   [5 6]]
```

Splitting

- `np.split(arr, num_sections, axis=...)` divides an array.

```
1 arr = np.arange(4) # [0 1 2 3]
2 part1, part2 = np.split(arr, 2) # >> [array([0, 1]), array([2, 3])]
```

Vectorization in Action: Element-wise Operations

This is the most direct example of vectorization.

Arithmetic operations are applied to each element without explicit loops.

```
1 a = np.array([10, 20, 30, 40])
2 b = np.array([1, 2, 3, 4])
3
4 # Vectorized arithmetic
5 print(a + b)  # >> [11 22 33 44]
6 print(a * b)  # >> [ 10  40  90 160]
7 print(a / 2)  # >> [ 5. 10. 15. 20.]
8
9 # Universal Functions (ufuncs) are also vectorized
10 print(np.sqrt(b)) # >> [1.  1.414  1.732  2. ]
```

This vectorized approach is much cleaner and faster than a Python for loop.

Linear Algebra: Two Kinds of Multiplication

Two fundamental ways to multiply arrays in NumPy:

1. Element-wise Product (*)

- A vectorized operation that multiplies elements at the same position. Shapes must be identical or broadcastable.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} a \times e & b \times f \\ c \times g & d \times h \end{bmatrix}$$

2. Matrix Dot Product (@)

- Standard matrix multiplication.
- Inner dimensions must match: $(m, n) @ (n, p) \rightarrow (m, p)$.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} @ \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} (ae + bg) & (af + bh) \\ (ce + dg) & (cf + dh) \end{bmatrix}$$

```
1 A = np.array([[1, 2], [3, 4]]); B = np.array  
  ([1, 1], [1, 1])  
2 print(A * B)    # Element-wise -> [[1 2], [3 4]]  
3 print(A @ B)    # Dot Product   -> [[3 3], [7 7]]
```

Broadcasting: Enabling Vectorization

Broadcasting describes how NumPy performs vectorized operations on arrays with different shapes.

Rule: If compatible, NumPy “stretches” the smaller array to match the shape of the larger one, without actually copying data.

```
1 matrix = np.array([[1, 2, 3],  
2                   [4, 5, 6]]) # Shape: (2, 3)  
3  
4 vector = np.array([10, 20, 30]) # Shape: (3,)  
5  
6 # NumPy "broadcasts" the vector to each row of the  
   matrix  
7 result = matrix + vector  
8 print(result)  
9 # [[11 22 33]  
10 #  [14 25 36]]
```

This is essential for common AI tasks, like adding a bias vector to a matrix of inputs.

Vectorized Aggregations: Summarizing Data

NumPy provides fast, vectorized functions to compute summaries of arrays, again avoiding slow Python loops.

```
1 data = np.array([[1, 2, 3], [4, 5, 6]])
2 # [[1 2 3]
3 #  [4 5 6]]
4
5 # Aggregate over the entire array
6 print(data.sum())      # >> 21
7 print(data.mean())    # >> 3.5
8 print(data.max())     # >> 6
9
10 # Aggregate along an axis
11 # axis=0 -> collapse rows (compute along columns)
12 print(data.sum(axis=0)) # >> [5 7 9]
13
14 # axis=1 -> collapse columns (compute along rows)
15 print(data.sum(axis=1)) # >> [ 6 15]
```

Advanced: Einstein Summation (einsum)

A powerful mini-language for defining tensor operations using index notation:

Syntax: 'sub1,sub2,...->out' where each operand has index labels.

Example: Matrix Multiplication

```
1 # 'ij,jk->ik' means:  
2 # Sum over the common index 'j'  
3 A = np.array([[1, 2], [3, 4]])  
4 B = np.array([[1, 1], [1, 1]])  
5 C = np.einsum('ij,jk->ik', A, B) # C is [[3,3],[7,7]]
```

Other examples show its power:

Transpose a matrix: `np.einsum('ij->ji', A)`

Sum along rows: `np.einsum('ij->i', A)`

Get the diagonal: `np.einsum('ii->i', A)`

■ Accessing Data: Indexing & Slicing

Similar to Python lists, but more powerful for multi-dimensional arrays.

Basic Indexing & Slicing

```
1 arr = np.arange(10) # [0 1 2 3 4 5 6 7 8 9]
2 print(arr[5])       # >> 5
3 print(arr[2:5])     # >> [2 3 4]
4 mat = np.array([[1,2,3], [4,5,6], [7,8,9]])
5 print(mat[1, 2])    # Element at row 1, col 2 >> 6
6 print(mat[:2, 1:])  # First 2 rows, from col 1 onwards
```

■ Accessing Data: Indexing & Slicing

Similar to Python lists, but more powerful for multi-dimensional arrays.

Boolean Indexing (Vectorized Selection)

- A powerful form of vectorized selection.
- Create a **boolean mask** to select elements that meet a condition, all without a loop

```
1 data = np.array([10, -5, 0, 20, -15])
2 print(data > 0)      # >> [True False False True False]
3 print(data[data > 0]) # >> [10 20]
```

Views vs. Copies: A Critical Concept

To save memory, NumPy avoids making copies whenever possible.

A Slice is a View

- A slice is a **view** into the original array's data.
- Modifying the view **will modify the original array!**

```
1 arr = np.arange(5)      # [0 1 2 3 4]
2 view_slice = arr[1:4]   # [1 2 3]
3 view_slice[0] = 99      # Modify the slice
4 print(arr)              # [0 99 2 3 4] <- Original is changed!
```



Views vs. Copies: A Critical Concept

To save memory, NumPy avoids making copies whenever possible.

Making an Explicit Copy

- Use the `.copy()` method to create a new, independent array.
- Most vectorized operations (arithmetic, boolean indexing) also create copies.

```
1 arr = np.arange(5)
2 copy_slice = arr[1:4].copy()
3 copy_slice[0] = 99
4 print(arr) # >> [0 1 2 3 4] <- Original is safe!
```

Accessing Data: Fancy Indexing

Use arrays of integers to access or modify elements at specific, non-contiguous positions. This creates a copy, not a view.

1D Example

```
1 arr = np.arange(20, 30) # [20, 21, ..., 29]
2 indices = np.array([0, 3, 8])
3 print(arr[indices]) # >> [20 23 28]
```

2D Example: Selecting specific rows/columns

- Common in AI for selecting a batch of data from a large dataset

```
1 matrix = np.arange(12).reshape(4, 3)
2 # [[ 0  1  2], [ 3  4  5], [ 6  7  8], [ 9 10 11]]
3
4 # Select rows 0 and 3
5 print(matrix[[0, 3]])
6 # >> [[ 0  1  2]
7        [ 9 10 11]]
8 # Select elements at (1, 1) and (3, 2)
9 print(matrix[[1, 3], [1, 2]]) # >> [4 11]
```

Vectorized Conditional Logic & Searching

Avoid Python loops for conditional logic by using vectorized NumPy functions.

Vectorized if-else: np.where()

- np.where(condition, value_if_true, value_if_false).

```
1 # Replace all negative numbers with 0
2 arr = np.array([10, -5, 0, 20, -15])
3 clipped_arr = np.where(arr < 0, 0, arr)
4 # >> [10  0  0 20  0]
```

Finding Indices of Top Elements: np.argsort()

- Returns the indices that would sort the array.
- Useful in AI for finding the predictions with the highest confidence..

```
1 # Model predictions (e.g., probabilities for 5 classes)
2 predictions = np.array([0.1, 0.8, 0.05, 0.0, 0.05])
3 # Get the indices of the top 2 predictions
4 top_2_indices = np.argsort(predictions)[-2:] # >> [0 1]
5 print(np.flip(top_2_indices)) # >> [1 0] (highest first)
```

Vectorized Set Operations for Feature Engineering

NumPy provides fast, vectorized set operations, which are useful for data exploration and creating features.

Finding Unique Elements

- `np.unique()` is fundamental for understanding categories in a feature.

```
1 feature = np.array(['cat', 'dog', 'cat', 'mouse', 'dog'])
2 unique_vals, counts = np.unique(feature, return_counts=True)
3 print(unique_vals) # >> ['cat' 'dog' 'mouse']
4 print(counts)      # >> [2 2 1]
```

Membership Testing

- `np.in1d()` tests if elements of one array are in another, returning a boolean mask.

```
1 # Create a feature: is the animal a common pet?
2 common_pets = ['cat', 'dog']
3 is_common_pet_mask = np.in1d(feature, common_pets)
4 print(is_common_pet_mask) # >> [ True  True  True False  True]
```

Reproducible Randomness for AI

Randomness is key for initializing weights, shuffling data, and augmentation. Reproducibility is essential for debugging.

The Modern Way: `np.random.Generator`

- Create a generator instance with a seed

```
1 # Create a generator with a seed for reproducibility
2 rng = np.random.default_rng(seed=42)
3
4 # Generate random numbers from a standard normal
   distribution
5 random_weights = rng.normal(size=(2, 3))
6
7 # Shuffle an array in-place
8 data_indices = np.arange(10)
9 rng.shuffle(data_indices) # Always the same with seed=42
```

Summary of Common Operations

Category	Common Functions & Methods
Creation	<code>np.array</code> , <code>np.arange</code> , <code>np.zeros</code> , <code>np.ones</code> , <code>np.linspace</code> , <code>np.random.rand</code>
I/O	<code>np.save</code> , <code>np.load</code> , <code>np.savez_compressed</code>
Manipulation	<code>.reshape</code> , <code>.T</code> , <code>.flatten</code> , <code>np.concatenate</code> , <code>np.vstack</code> , <code>np.hstack</code> , <code>np.split</code>
Math (ufuncs)	<code>np.add</code> , <code>np.subtract</code> , <code>np.multiply</code> , <code>np.divide</code> , <code>np.sqrt</code> , <code>np.exp</code> , <code>np.sin</code>
Aggregation	<code>.sum</code> , <code>.mean</code> , <code>.std</code> , <code>.min</code> , <code>.max</code> , <code>.argmin</code> , <code>.argmax</code>
Linear Algebra	<code>np.dot</code> , <code>@</code> , <code>np.einsum</code> , <code>np.linalg.inv</code> , <code>np.linalg.eig</code>
Searching/Logic	<code>np.where</code> , <code>np.argsort</code> , <code>np.unique</code> , <code>np.in1d</code>
Indexing	Slicing <code>[:]</code> , boolean masking <code>[arr > 0]</code> , fancy indexing <code>[indices]</code>

Table: This is not exhaustive but covers the vast majority of common use cases.

In-Class Practice (Submit to Canvas)

Let's find the first Principal Component using vectorized operations.

- Generate a random 100×3 dataset (samples $\times$ features) using `np.random.default_rng()`.
- Normalize each feature (subtract mean, divide by std) via vectorized operation
- Compute the covariance matrix using `np.einsum`: $C = \frac{1}{n-1} X.T @ X$
- Use `np.linalg.eig` to find eigenvalues/eigenvectors of `C`.
- Get the index of the largest eigenvalue using the vectorized `np.argmax()`.
- Use this index to select the leading eigenvector (the first principal component).



SciPy

Problem: NumPy provides the essential ndarray for data representation, but it lacks the higher-level algorithms needed for model development and analysis.

- How do you find the optimal parameters that **minimize a model's loss function**?
- How do you perform a **statistical test** to compare the performance of two models?
- How do you efficiently handle the massive, yet sparse, datasets common in NLP?

The Solution: SciPy (Scientific Python)

- Built on **NumPy**, adds key algorithms for science and engineering.
- Provides ready-to-use tools to implement, optimize, and evaluate models.
- Handles complex numerical details so you can focus on AI logic.

The SciPy Ecosystem: Key AI Modules

SciPy is organized into sub-packages. For AI, a few stand out as particularly critical. You import the specific module you need, which keeps your code clean and efficient.

- `scipy.optimize`
- `scipy.stats`
- `scipy.linalg`
- `scipy.sparse`
- `scipy.special`
- `scipy.spatial`
- `scipy.fft`
- `scipy.io`

```
1 from scipy import optimize
2 from scipy import stats
```

Optimization: Minimizing Loss (scipy.optimize)

This is arguably the **most important SciPy module** for AI.

It finds the parameters that minimize a given objective function (i.e., a loss function).

Example: Minimizing a Simple Loss Function

- Imagine a simple loss function $L(w) = (w - 3)^2 + 5$.
- We want to find the weight w that minimizes this loss.

```
1 from scipy.optimize import minimize
2 import numpy as np
3 # Define a simple convex loss function
4 def loss_function(w):
5     return (w - 3)**2 + 5
6 # Provide an initial guess for the parameter w
7 initial_w = 0.0
8 # Run the minimization algorithm
9 result = minimize(loss_function, initial_w)
10 print(f"Optimal parameter w: {result.x[0]:.4f}") # >> 3.0000
11 print(f"Minimum loss value: {result.fun:.4f}")  # >> 5.0000
```

The optimizer correctly finds that $w = 3$ minimizes the function, a direct analogue to training a simple model.

AI Utilities (scipy.special & scipy.spatial)

Some modules provide highly specialized, efficient functions that are building blocks for AI algorithms.

scipy.special: Numerical Stability

- Provides numerically stable versions of common functions like softmax or logsumexp, preventing overflow/underflow errors with large inputs (logits).

```
1 from scipy.special import softmax
2 logits = np.array([1000, 1001, 1002])
3 # np.exp(logits) would cause an overflow warning
4 stable_probs = softmax(logits)
5 print(stable_probs) # >> [0.09 0.24 0.66]
```

scipy.spatial: Distance Metrics

- Crucial for algorithms like k-NN, clustering, and similarity searches.
- distance.cdist computes the distance between all pairs of points in two matrices, fast.

Advanced Linear Algebra (scipy.linalg)

scipy.linalg offers high-performance linear algebra routines, essential for many classical ML algorithms.

Example: Solving for Regression Weights

- The problem $Ax = b$ is the core of Ordinary Least Squares (Linear Regression), where we solve for the weights x

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 11 \end{bmatrix}$$

```
1 from scipy import linalg
2 A = np.array([[1, 2], [3, 4]])
3 b = np.array([5, 11])
4 x = linalg.solve(A, b)
5 print(f"Solution x (weights): {x}") # >> Solution x: [1. 2.]
6 # Verification: A @ x should equal b
7 print(f"A @ x = {A @ x}") # >> A @ x = [ 5. 11.]
```

Using `linalg.solve` is numerically stable and faster than manually computing the matrix inverse.

Statistics: Comparing Models (scipy.stats)

A comprehensive module for statistical analysis.

A key use case in AI is determining if the performance difference between two models is statistically significant.

Example: Two-sample t-test for Model Accuracy

- We have cross-validation accuracy scores for two models.
- Is Model B actually better, or is the difference due to chance? The p-value is key.

```
1 from scipy import stats
2 # 10 CV scores for Model A (e.g., accuracy)
3 model_a_scores = np.random.normal(loc=0.85, scale=0.05, size=10)
4 # 10 CV scores for Model B (with a slightly higher mean)
5 model_b_scores = np.random.normal(loc=0.88, scale=0.05, size=10)
6 t_statistic, p_value = stats.ttest_ind(model_a_scores, model_b_scores)
7 print(f"P-value: {p_value:.4f}") # e.g., P-value: 0.1588
8 if p_value < 0.05:
9     print("The performance difference is statistically significant.")
10 else:
11     print("No significant performance difference was found.")
```

A small p-value (< 0.05) suggests the improvement is real

Working with Sparse Data (scipy.sparse)

In many AI domains like NLP (Bag-of-Words/TF-IDF) or Recommender Systems (user-item interactions), matrices are enormous but mostly zeros.

Storing them as dense NumPy arrays is infeasible

Example: Compressed Sparse Row (CSR) Format

- Stores only the non-zero elements and their indices, saving vast memory.

```
1 from scipy import sparse
2 # A mostly-zero matrix (e.g. a tiny doc-term matrix)
3 M_dense = np.array([[1, 0, 0, 0, 5], [0, 0, 2, 0, 0], [0, 8, 0, 3,
      0]])
4 # Convert to a memory-efficient sparse representation
5 M_sparse = sparse.csr_matrix(M_dense)
6 print(M_sparse)
7 # (0, 0)      1
8 # (0, 4)      5
9 # (1, 2)      2
10 # (2, 1)      8
11 # (2, 3)      3
12 print(f"Memory (dense): {M_dense.nbytes} bytes")
13 print(f"Memory (sparse): {M_sparse.data.nbytes + M_sparse.indices.
      nbytes + M_sparse.indptr.nbytes} bytes")
```

Summary of SciPy Modules for AI

Module	Common AI Use Cases
<code>scipy.optimize</code>	Minimizing model loss functions (<code>minimize</code>), fitting curves
<code>scipy.stats</code>	Hypothesis tests for model comparison (<code>ttest_ind</code>), probability distributions
<code>scipy.linalg</code>	Solving for model weights (<code>solve</code>), matrix decompositions (<code>svd</code>) for PCA
<code>scipy.sparse</code>	Representing text data (TF-IDF) or user-item interaction matrices
<code>scipy.special</code>	Numerically stable activation functions (<code>softmax</code> , <code>logsumexp</code>)
<code>scipy.spatial</code>	Fast distance calculations for k-NN and clustering (<code>distance.cdist</code>)

Table: SciPy provides the high-level algorithms that complement NumPy's array tools.

Apply Your SciPy Knowledge.

Optimization:

- Find the minimum value of the function $f(x) = (x - 5)^2 + \cos(x)$.
- Use `scipy.optimize.minimize` with an initial guess of $x_0 = 0$.
- Does it find the global minimum?

Numerical Stability:

- A model outputs logits $z = \text{np.array}([500, 505, 502])$.
- Calculate the probability distribution using `scipy.special.softmax`.
- Why would a naive implementation using `np.exp(z)` be a bad idea?

Linear Algebra:

- Solve the following system of equations for w_1, w_2 using `scipy.linalg.solve`:

$$\begin{cases} 3w_1 + 2w_2 = 12 \\ w_1 + 4w_2 = 14 \end{cases}$$



Pandas

Why Pandas?

Problem: Raw data is messy. AI models require clean, well-structured numerical data. NumPy are efficient but aren't designed for data loading, cleaning, and manipulation.

- How do you load and explore a dataset with mixed data types (text, numbers, categories)?
- How do you handle the missing values common in real-world data?
- How do you engineer new features to improve your model's performance?

The Solution: Pandas (Python Data Analysis Library)

- The cornerstone of the AI/ML data preprocessing pipeline in Python.
- Provides the DataFrame, an intuitive, powerful structure for handling tabular data.
- Makes feature engineering, data cleaning, and exploratory data analysis (EDA) straightforward

The Core: The Pandas Series (1D)

A **Series** is a one-dimensional array-like object that can hold any data type and has an associated array of data labels, called its **index**

Think of it as a single column in a spreadsheet.

```
1 import pandas as pd
2 import numpy as np # Often used with pandas
3 s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
4
5 print(s)
6 # a      10
7 # b      20
8 # c      30
9 # dtype: int64
```

You can access data via the index label (e.g., `s['b']`) or by integer position (`s.iloc[1]`).
Operations are aligned by index.

The Workhorse: The Pandas DataFrame (2D)

A DataFrame is a two-dimensional, size-mutable, tabular data structure with labeled axes (rows and columns)

- It's like a spreadsheet or an SQL table.
- You can think of it as a dictionary of Series objects

```
1 data = {'State': ['CA', 'TX', 'FL', 'NY'],  
2         'Population': [39.2, 29.5, 21.7, 19.3]}  
3 df = pd.DataFrame(data)  
4  
5 print(df)  
6 #    State  Population  
7 # 0    CA         39.2  
8 # 1    TX         29.5  
9 # 2    FL         21.7  
10 # 3    NY         19.3
```

The DataFrame is the primary object you will use for most data analysis tasks in Pandas.

Getting Data In & Out: I/O

A key strength of Pandas is its ability to easily read and write data from many formats.

Key Function: `pd.read_csv()`

- The most common way to load data. It's powerful and has many options to handle different file formats.

```
1 # Assume we have a file 'data.csv'
2 # Name, Age, City
3 # Alice, 25, NYC
4 # Bob, 30, LA
5 df = pd.read_csv('data.csv')
6 print(df)
7 #      Name  Age  City
8 # 0  Alice   25  NYC
9 # 1   Bob   30   LA
10 # To save a DataFrame back to CSV:
11 df.to_csv('output.csv', index=False)
```

Pandas also has readers for Excel (`read_excel`), SQL (`read_sql`), JSON (`read_json`), and more.

Accessing Data: Indexing & Selection

Pandas provides powerful ways to select subsets of your data.

Key Methods: `.loc[]` and `.iloc[]`

- `df.loc[]`: Selects data by **label** (index/column names).
- `df.iloc[]`: Selects data by **integer position**.
- **Boolean Indexing**: Filter data based on a condition.

```
1 data = {'State': ['CA', 'TX', 'FL', 'NY'],
2         'Population': [39.2, 29.5, 21.7, 19.3]}
3 df = pd.DataFrame(data)
4 # Using the population DataFrame from before
5 # Select a column
6 print(df['Population']) # Returns a Series
7 # Boolean indexing
8 print(df[df['Population'] > 25])
9 #    State  Population
10 # 0    CA         39.2
11 # 1    TX         29.5
```

Applying Custom Functions: `.apply()`

For more complex, custom transformations, the `.apply()` method is invaluable.

It applies a function along an axis of the DataFrame.

This is a key tool for feature engineering

```
1 # Create a DataFrame
2 data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Score': [85, 95, 60]}
3 df_scores = pd.DataFrame(data)
4 # Define a function to apply
5 def get_grade(score):
6     if score >= 90:
7         return 'A'
8     elif score >= 80:
9         return 'B'
10    else:
11        return 'C'
12 # Create a new 'Grade' column by applying the function
13 df_scores['Grade'] = df_scores['Score'].apply(get_grade)
14 print(df_scores)
15 #      Name  Score Grade
16 # 0   Alice    85     B
17 # 1    Bob    95     A
18 # 2  Charlie    60     C
```

Cleaning Data: Handling Missing Values

Real-world data is often incomplete.

Pandas represents missing data with NaN (Not a Number) and provides tools to handle it.

```
1 data = {'A': [1, 2, np.nan], 'B': [5, np.nan, np.nan]}
2 df_missing = pd.DataFrame(data)
3 print(df_missing)
4 #      A      B
5 # 0  1.0  5.0
6 # 1  2.0  NaN
7 # 2  NaN  NaN
8 # Fill missing values with the mean of the column
9 df_filled = df_missing.fillna(df_missing.mean())
10 print(df_filled)
11 #      A      B
12 # 0  1.0  5.0
13 # 1  2.0  5.0
14 # 2  1.5  5.0
```

You can also drop rows with missing data using `df.dropna()` or drop entire columns using `df.drop(columns=['B'])`.

Combining DataFrames: Concat & Merge

Often, your data is spread across multiple files or tables.
Pandas provides high-performance tools to combine them

Concatenating (pd.concat): Stacking dataframes together (either vertically or horizontally).

```
1 df1 = pd.DataFrame({'A': ['A0', 'A1'], 'B': ['B0', 'B1']})
2 df2 = pd.DataFrame({'A': ['A2', 'A3'], 'B': ['B2', 'B3']})
3 result = pd.concat([df1, df2]) # Stacks df2 below df1
4 print(result)
5 #      A  B
6 # 0  A0  B0
7 # 1  A1  B1
8 # 0  A2  B2
9 # 1  A3  B3
```

Combining DataFrames: Concat & Merge

Often, your data is spread across multiple files or tables. Pandas provides high-performance tools to combine them

Merging (pd.merge): SQL-style joins on a common key.

```
1 left = pd.DataFrame({'key': ['K0', 'K1'], 'val1': [0, 1]})
2 right = pd.DataFrame({'key': ['K0', 'K1'], 'val2': [2, 3]})
3 merged = pd.merge(left, right, on='key')
4 print(merged)
5 #    key  val1  val2
6 # 0  K0     0     2
7 # 1  K1     1     3
```

■ The Powerhouse: GroupBy Operations

The "group by" mechanism allows you to **split** data into groups, **apply** a function to each group, and **combine** the results.

```
1 data = {'Team': ['A', 'B', 'A', 'B', 'A'],
2         'Points': [10, 12, 12, 15, 13]}
3 df_team = pd.DataFrame(data)
4
5 # Group by team and calculate the sum of points for each
6 team_scores = df_team.groupby('Team').sum()
7 print(team_scores)
8 #      Points
9 # Team
10 # A      35
11 # B      27
```

`df.groupby("X")["Y"].operation()`

This is extremely powerful for summarizing data and is a cornerstone of data analysis.

Summary of Common Pandas Operations

Category	Example Functions & Purpose
I/O	<code>pd.read_csv</code> , <code>df.to_csv</code> : Read from and write to files.
Selection	<code>df['col']</code> , <code>df.loc[]</code> , <code>df.iloc[]</code> : Select columns, rows, or specific data points.
Cleaning	<code>df.dropna()</code> , <code>df.fillna()</code> , <code>df.drop()</code> : Handle missing data or remove data.
Transformation	<code>df.apply()</code> : Apply a custom function to data.
Combining	<code>pd.merge</code> , <code>pd.concat</code> , <code>df.join()</code> : Combine multiple DataFrames.
Grouping	<code>df.groupby('key').sum()</code> : Split-apply-combine operations for aggregation.
Statistics	<code>df.describe()</code> , <code>df.mean()</code> , <code>df.corr()</code> : Compute summary statistics.

Table: Pandas provides a comprehensive toolkit for the entire data analysis workflow.

Apply Your Pandas Knowledge.

Creation & Merging:

- Create a 'grades' DataFrame: 'Name', 'Subject', 'Score'. (5+ rows)
- Create a 'students' DataFrame: 'Name', 'Major'.
- Merge the two DataFrames together into one

Filtering:

- Select all the rows where the 'Subject' is 'Math'.
- Then, select all rows where the 'Score' was greater than 90

Transformation:

- Use `.apply()` on the 'Score' column to create a new 'Pass/Fail' column (e.g., score $\geq$ 70 is 'Pass'):

Aggregation:

- Use `groupby()` to find the average score for each subject.
- Use `groupby()` to find the average score for each Major.



Matplotlib & Seaborn

■ Why Matplotlib?

Problem: In AI/ML we work with high-dimensional data, model weights, and metrics. Raw arrays/dataframes are hard to interpret.

- Exploratory Data Analysis (EDA): Feature distributions, outliers, correlations?
- Training: Is the model learning or overfitting?
- Evaluation: How well does it perform? Which classes are confused?
- Debugging: Why strange predictions? What has been learned?

The Solution: Matplotlib & Seaborn

- Matplotlib: Core library for static, animated, interactive plots.
- Seaborn: Built on Matplotlib, makes statistical graphics simple.
- Visualization is the first and most important step in any AI project.

■ Your First AI Plot: Tracking Model Training

The most common plot in deep learning is the loss curve.

We'll start with the simple pyplot interface.

```
1 import matplotlib.pyplot as plt
2 # 1. Sample Training Data (e.g., from Keras/PyTorch)
3 epochs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4 loss = [0.8, 0.6, 0.45, 0.35, 0.28, 0.22, 0.18, 0.15, 0.12, 0.10]
5 val_loss = [0.9, 0.75, 0.6, 0.5, 0.45, 0.41, 0.39, 0.38, 0.37, 0.36]
6 # 2. Create Plot
7 plt.plot(epochs, loss, 'bo-', label='Training Loss')
8 plt.plot(epochs, val_loss, 'ro--', label='Validation Loss')
9 # 3. Add Labels, Title, and Legend
10 plt.title("Model Training & Validation Loss")
11 plt.xlabel("Epochs")
12 plt.ylabel("Loss")
13 plt.grid(True)
14 plt.legend()
15 # 4. Show Plot
16 plt.show()
```

■ The Pro's Choice: Object-Oriented Interface

For more control, especially with multiple plots, the Object-Oriented (OO) API is superior. This is the standard for complex AI visualizations.

- **Figure:** The entire window or page the plot is drawn on.
- **Axes:** An individual plot or chart within the Figure.

```
1 # Data from previous slide...
2 # 2. Create Figure and a single Axes object
3 fig, ax = plt.subplots(figsize=(8, 5))
4 # 3. Plot on the Axes object
5 ax.plot(epochs, loss, 'bo-', label='Training Loss')
6 ax.plot(epochs, val_loss, 'ro--', label='Validation Loss')
7 # Use `set_*` methods for labels and title
8 ax.set_title("Model Loss (OO Style)")
9 ax.set_xlabel("Epochs")
10 ax.set_ylabel("Loss")
11 ax.grid(True)
12 ax.legend()
13 # 4. Show Plot (or save with fig.savefig())
14 plt.show()
```

Feature Distributions with Histograms

Understanding how your data is distributed is crucial for feature engineering and model selection.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # Generate skewed data (e.g., feature like 'income')
4 data = np.random.gamma(2, 2, 1000)
5 fig, ax = plt.subplots()
6 # Plot a histogram to see the distribution
7 ax.hist(data, bins=50, density=True, alpha=0.7, label='
    Histogram')
8 ax.set_title("Distribution of a Skewed Feature")
9 ax.set_xlabel("Feature Value")
10 ax.set_ylabel("Density")
11 plt.show()
```

This plot immediately tells us the feature is not normally distributed, which might inform a decision to apply a log transform.

Visualizing Image Data with imshow

For computer vision tasks, imshow is essential for displaying images, filters, and feature maps.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 cnn_filter = np.random.randn(7, 7)
4 # For this example, we create a synthetic one
5 digit = np.array([
6     [0, 0, 5, 13, 9, 1, 0, 0], [0, 0, 13, 15, 10, 15, 5, 0],
7     [0, 3, 15, 2, 0, 11, 8, 0], [0, 4, 12, 0, 0, 8, 8, 0],
8     [0, 5, 8, 0, 0, 9, 8, 0], [0, 4, 11, 0, 1, 12, 7, 0],
9     [0, 2, 14, 5, 10, 12, 0, 0], [0, 0, 6, 13, 10, 0, 0, 0]
10 ])
11 # Create a 1x2 subplot figure
12 fig, axes = plt.subplots(1, 2, figsize=(10, 5))
13 # Display the filter
14 im1 = axes[0].imshow(cnn_filter, cmap='viridis')
15 axes[0].set_title('CNN Filter Visualization')
16 fig.colorbar(im1, ax=axes[0])
17 # Display the digit
18 im2 = axes[1].imshow(digit, cmap='gray')
19 axes[1].set_title('MNIST-like Digit')
20 fig.colorbar(im2, ax=axes[1])
21 plt.show()
```

Relationships with Scatter Plots

Scatter plots help us find relationships between two features and see if they are separable by class. Seaborn makes this easy.

```
1 import seaborn as sns
2 from sklearn.datasets import load_iris
3 # Load a classic dataset
4 iris = load_iris()
5 X = iris.data
6 y = iris.target
7 fig, ax = plt.subplots(figsize=(8, 6))
8 # Use Seaborn's scatterplot for easy coloring by class
9 sns.scatterplot(
10     x=X[:, 0], y=X[:, 1], # Sepal Length vs Sepal Width
11     hue=iris.target_names[y],
12     palette='deep',
13     ax=ax
14 )
15 ax.set_title("Iris Dataset: Sepal Length vs. Width")
16 ax.set_xlabel("Sepal Length (cm)")
17 ax.set_ylabel("Sepal Width (cm)")
18 plt.show()
```

Model Evaluation: The Confusion Matrix

A confusion matrix is the best way to see a classifier's performance. Seaborn's heatmap is perfect for this.

```
1 import numpy as np
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 # Example Confusion Matrix (Actual vs. Predicted)
5 # Rows: Actual Class, Columns: Predicted Class
6 cm_data = np.array([[120, 5, 2],
7                     [8, 110, 1],
8                     [1, 3, 95]])
9 labels = ['Cat', 'Dog', 'Bird']
10 fig, ax = plt.subplots(figsize=(7, 5))
11 sns.heatmap(cm_data, annot=True, fmt='d', cmap='Blues',
12             xticklabels=labels, yticklabels=labels, ax=ax)
13 ax.set_title("Confusion Matrix")
14 ax.set_xlabel("Predicted Label")
15 ax.set_ylabel("True Label")
16 plt.show()
```

This clearly shows the model is good but sometimes confuses cats for dogs (8 instances).

Category	Function(s)	Common AI Use Case
Setup	<code>fig, ax = plt.subplots()</code>	Standard way to start any complex plot.
Line Plots	<code>ax.plot()</code>	Plotting training/validation loss & accuracy curves.
Scatter Plots	<code>ax.scatter()</code> , <code>sns.scatterplot()</code>	Visualizing feature relationships, model predictions vs. actuals.
Distributions	<code>ax.hist()</code> , <code>sns.histplot()</code> , <code>sns.kdeplot()</code>	Understanding feature distributions for preprocessing.
Matrix/Image	<code>ax.imshow()</code> , <code>sns.heatmap()</code>	Displaying images, feature maps, attention, confusion matrices.
Labels	<code>ax.set_title()</code> , <code>ax.set_xlabel()</code>	Labeling plots for reports and presentations.
Legends	<code>ax.legend()</code>	Differentiating metrics (e.g., train vs. validation).
Output	<code>plt.show()</code> , <code>fig.savefig()</code>	Displaying plots or saving them for publications.

Table: A toolkit for common visualization tasks in Machine Learning.

In-Class Practice

AI Visualization Challenges.

Loss Curve:

- Plot the training and validation accuracy curves below.
- Make training a solid blue line and validation a dashed red line.
- Add a title, labels, and a legend.

```
epochs = range(1, 11)
acc = [0.75, 0.82, 0.85, 0.88, 0.90, 0.91, 0.92,
       0.93, 0.93, 0.94]
val_acc = [0.78, 0.81, 0.84, 0.86, 0.87, 0.87, 0.88,
           0.88, 0.87, 0.88]
```

Feature Scatter:

- Using the Iris dataset from a previous slide, create a scatter plot of 'Petal Length' (column 2) vs. 'Petal Width' (column 3).
- Use seaborn to color the points by species

Confusion Matrix:

- A model produces the 2x2 confusion matrix below for a binary classification task ('Negative', 'Positive'). Visualize it using seaborn.heatmap

```
cm = [[250, 15], [30, 205]] # [[TN, FP], [FN, TP]]
```

NumPy: Fast array operations, vectorization, broadcasting, linear algebra.

SciPy: Optimization, statistics, sparse data, advanced linear algebra.

Pandas: DataFrames for cleaning, transforming, combining, summarizing data.

Matplotlib/Seaborn: Visualization for exploration, debugging, evaluation.

Together these form the Python ecosystem core for AI/ML



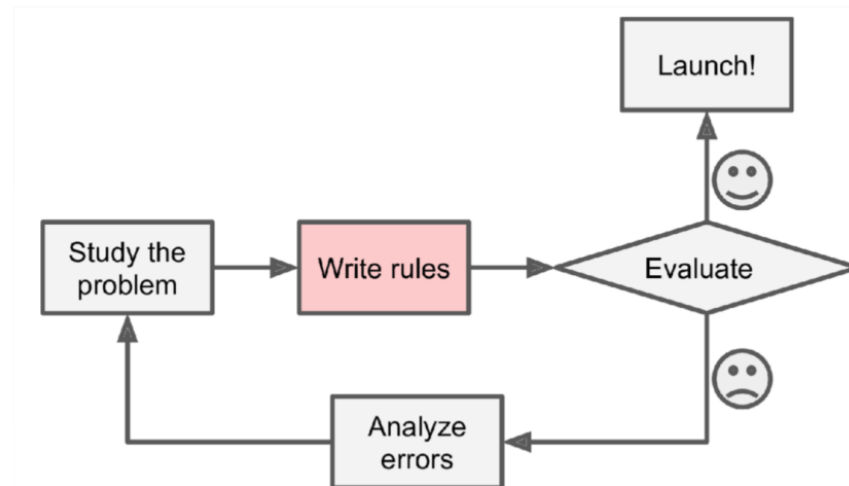
Machine Learning Basics

Why Use Machine Learning?

Let's consider building a spam filter.

Traditional Approach

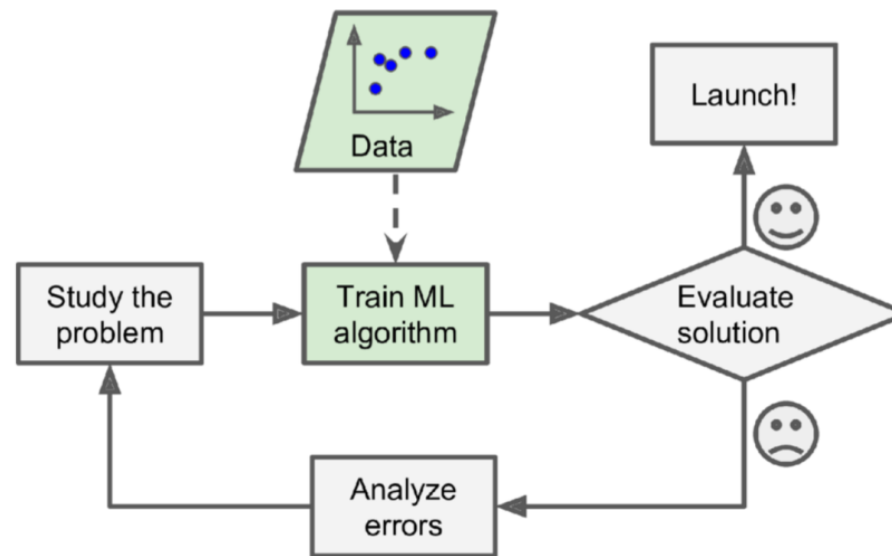
- Study the problem and identify patterns (e.g., words like "free", "credit card").
- Write explicit rules for each pattern.
- Evaluate, analyze errors, and refine the rules.
- This process is complex and hard to maintain



The traditional programming approach is based on hand-crafted rules

Why Use Machine Learning?

- In contrast, an ML-based spam filter automatically learns which words and phrases are good predictors of spam by observing examples.
- The program is often shorter, easier to maintain, and more accurate.
- The system learns from a training set of examples.



The ML approach learns from data to find a solution.

Arthur Samuel (1959)

Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.

Tom Mitchell (1997)

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

■ Core Terminology: The Data & Process

- Sample / Instance: A single row in your dataset (e.g., one house, one email).
- Features (X): The input variables (columns) used for prediction.
- Label (y): The output variable you are trying to predict.
- Model: The algorithm that learns the mapping from features X to the label y.
- Training: The process where the model "learns" from a dataset.
- Inference: Using the trained model to make a prediction on a new sample

Numerical Data

Data that represents quantities.

- **Continuous:** Can take any value within a range (e.g., temperature, height, price).
- **Discrete:** Can only take specific integer values (e.g., number of bedrooms, age in years).

Categorical Data

Data that represents qualitative groups.

- **Nominal:** Categories with no intrinsic order (e.g., color: Red/Green/Blue, city: Beijing/London).
- **Ordinal:** Categories with a meaningful order (e.g., education level: High School/Bachelor's/Master's).

ML systems can be classified based on the type and amount of supervision they receive during training.

The four main categories are:

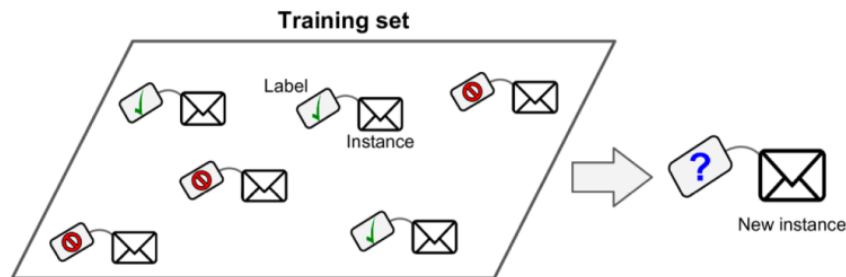
- **Supervised** Learning
- **Unsupervised** Learning
- **Semi-supervised** Learning
- **Reinforcement** Learning

Supervised Learning

In supervised learning, the training data is a set of n samples $\{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$ where each sample includes a feature vector $\mathbf{x}$ and its corresponding label y . The goal is to learn a mapping function f such that $f(\mathbf{X}) \approx \mathbf{y}$.

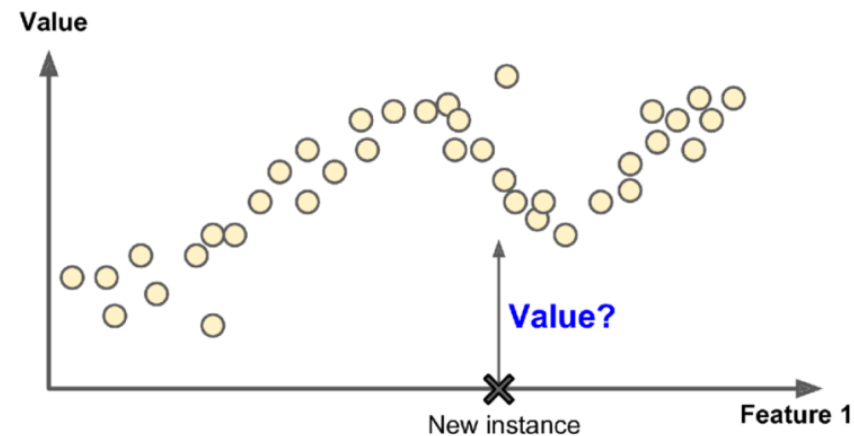
Classification

- The label y is from a finite set of discrete classes.
- Example: Is this email spam ($y = 1$) or not ($y = 0$)?



Regression

- The label y is a continuous numerical value
- Example: What is the price of this car ($y = 25000$)?

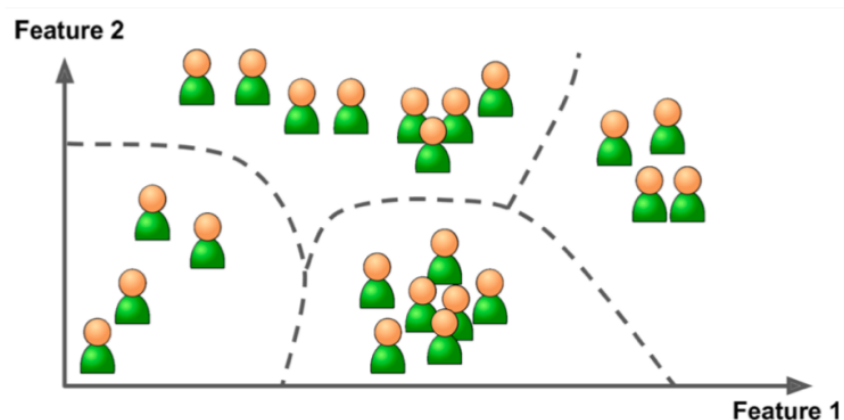


Unsupervised Learning

In unsupervised learning, the training data is a set of n samples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$ consisting only of input features **$\mathbf{X}$** **without any corresponding labels $\mathbf{y}$** . The system tries to learn the underlying structure or distribution in the data.

Clustering

- Goal: Group similar instances together based on their features $\mathbf{x}$.
- Example: Customer segmentation based on purchasing behavior.



Anomaly Detection

- Goal: Find instances $\mathbf{x}$ that are significantly different from the rest.
- Example: Fraud detection in credit card transactions.



- Deals with data that is partially labeled: many unlabeled instances and a few labeled ones.
- Useful because labeling data can be expensive and time-consuming.
- Example: Google Photos identifies a person in a few photos, and can then recognize them in all other photos.

- The learning system, called an agent, observes the environment, selects and performs actions, and gets rewards or penalties in return.
- It must then learn by itself what is the best strategy, called a policy, to get the most reward over time.
- Example: Training a robot to walk or a program to play a game like Go.
- Note: Reinforcement Learning is crucial for modern Large Language Models (LLMs), a topic we will cover in detail later.

The Core Machine Learning Workflow

Data Exploration and Preparation

- The first step is to study the dataset, which includes cleaning, analyzing its structure, and preparing it for the model.

Model Selection

- Choose a suitable learning algorithm that fits the problem, for example, Linear Regression for a regression task or a classifier for a classification task.

Training and Evaluation

- The model learns from the training data.
- Its performance is then assessed using appropriate metrics to see how well it has learned.

Prediction and Application

- Finally, the trained model is applied to make predictions on new, unseen cases

The two main things that can go wrong are

“bad algorithm” and “bad data”

Insufficient Quantity of Training Data

- Even for simple problems, you typically need thousands of examples.
- For complex problems like image or speech recognition, you may need millions.

Non-representative and Poor-Quality Data

- Your training data must be representative of the new cases you want to generalize to.
- Poor quality data due to errors, outliers, and noise will make it harder for the system to detect underlying patterns

Irrelevant Features

- "Garbage in, garbage out."
- The system will only be as good as the features it is trained on.
- Feature engineering is a critical step

■ Main Challenges: Bad Algorithm

There is no single best algorithm for all problems.

Choosing the right one involves understanding various trade-offs.

Trade-off 1: Scalability vs. Speed

- For linear regression, the Normal Equation is fast on datasets with a small number of features, but Gradient Descent scales better to large numbers of features.
- Algorithms like Stochastic and Mini-batch GD are much faster than Batch GD on large datasets and support out-of-core learning.

Trade-off 2: Model Assumptions vs. Data Structure

- Linear models assume the data is linear; choosing them for complex, non-linear data will lead to poor performance (underfitting).
- K-Means clustering performs poorly on non-spherical clusters or clusters of varying sizes and densities. A density-based algorithm like DBSCAN would be a better choice for such data.

Libraries for AI Workflows

	Library	Core Idea	Primary Data Type	Typical Tasks
• Numpy	NumPy	Vectorized numerical computing	ndarray	Element-wise ops, broadcasting, LA, random
• SciPy	SciPy	Algorithms on arrays	ndarray/sparse	Optimize, stats tests, sparse ops, FFT
• Pandas	pandas	Tabular data wrangling	DataFrame/Series	I/O, cleaning, merge/join, groupby, features
	Matplotlib	Visualization backend	n/a	Line/scatter/bar, images, layout, export
• Matplotlib & Seaborn				

Machine Learning Basics

- Core Terminology
- Main Categories: Supervised/UnsupervisedSemi-supervised/Reinforcement Learning
- Core Workflow: Data, Model, Train&Eval, Predict
- Main Challenges: “bad algorithm” and “bad data”

Python Programming with Artificial Intelligence

Thank you for listening!

